



1. Data Preparation for Machine Learning

Every model starts with data. NO EXCEPTION!!

Previous Section:

 [0. Machine Learning](#)

Contents:

[1. Missing Values in Data](#)

[2. Training Data and Testing Data](#)

[3. Validation Data — “The Mock Exam for the Model”](#)

[Summary](#)

[References](#)

Before any model learns, before any prediction is made, before any “intelligence” appears—there is one thing that quietly controls everything: **Data**.

Not the model. Not the algorithm. Not the fancy math → It is *Data*

Every Machine Learning system—no matter how advanced—starts from the same place: a dataset. Rows representing observations, columns representing features, and somewhere in that structure, the signal we are trying to uncover.

But here’s something beginners often misunderstand. They think: ***“Once I pick a model... I’m good”***. No. Not even close. Because a model is only as good as the data you feed it.

- If the data is messy → the model becomes confused
- If the data is incomplete → the model becomes biased
- If the data is inconsistent → the model becomes unreliable

And this leads us to something that is often overlooked, yet absolutely critical: **Data Preparation**. You might hear it called: Data Preprocessing; Data Cleaning; Data Analysis; Data Manipulation.

Different names, same idea. It is the process of turning raw, unstructured, imperfect data into something a machine can actually learn from.

Because let's be honest, real-world data is never clean. It comes with: Missing values; Noise; Outliers; Inconsistent formats; Mixed data types. And if you try to train a model directly on that? It doesn't "learn". It struggles.

So before we even think about training, we must prepare the data. We clean it, transform it, organize it, standardize it. Only then does learning begin.

And here's the important shift:

***Data Preparation is not a side task. It is not optional. It is not "just preprocessing."
→ It is the foundation of Machine Learning.***

In fact, in real-world projects, this step often takes **more time than building the model itself**.

Why? Because models are predictable. Data is not. So in this section, we step into that reality. Not the clean textbook version, but the actual process of dealing with data.

And we begin with one of the most common—and most important—problems you will ever face: Missing Values.

1. Missing Values in Data

Let's talk about something that *quietly breaks models* more often than anything else: **Missing values**.

They look harmless. Just a few "NaN" ("Not a Number") here, a blank cell there, maybe a "?" or "Unknown". But to a Machine Learning model? That's not "missing" → That's **confusing**

Because the model doesn't understand absence. It only understands numbers. So before we do anything meaningful with data, we must answer a simple question:

"What do we do when information is missing?"

The Reality of Missing Data - In real-world datasets, missing values are everywhere:

- A customer didn't enter their age
- A sensor failed to record a value
- A user skipped a question
- A system logged incomplete data

And here's the key insight: **Missing data is not just empty space. It carries meaning.** Sometimes it's random. Sometimes it's systematic. Sometimes, it tells you something important. So we don't just "fill it blindly". We **impute** it.

What is Imputation? Imputation is the process of replacing missing values with something reasonable... something that allows the model to continue learning.

There is no single "correct" way. Only choices and trade-offs. Let's walk through the most common ones.

- **Mean / Median / Mode Imputation:** This is the most straightforward approach. We replace missing values using statistical summaries of the column.
 - **Mean** → best for continuous, normally distributed data
 - **Median** → better for skewed data (robust to outliers)
 - **Mode** → useful for categorical or discrete values

Simple. Fast. Effective. But there's a catch: It reduces variability. It can distort the true distribution. Still, it's often the first tool you reach for.

- **Forward / Backward Fill:** This method borrows information from nearby data.
 - **Forward fill** → use the previous value
 - **Backward fill** → use the next value

This works especially well in **time-series data**, where order matters. Think of stock prices, sensor readings, or daily measurements. Because in those cases, the past often tells you something about the present.

- **Constant Value Imputation:** Sometimes, missing data isn't just "unknown"... it's a category of its own. So we replace it with a constant like "Unknown" ; "Not Available" or a specific marker like 1 → This is useful when **missingness itself is meaningful**.
- **Interpolation:** Now we move into something a bit smarter.

Instead of inserting a fixed value, we estimate it based on surrounding values. Linear interpolation. Polynomial interpolation. This works well when the data has a **trend**. You're not guessing randomly. You're following the pattern.

- **Model-Based Imputation:** Now we step into more advanced territory. Instead of using simple statistics, we train a model to predict the missing values.
 - Use other features → Learn relationships → Predict the missing entry

This can be powerful. But also more complex, more computationally expensive, easier to misuse if not careful

- **Deletion (The Tempting Shortcut):** The simplest solution, you *can* just drop missing data. Remove rows Remove columns. And sometimes, that's acceptable. But be careful. Because every deletion is a loss of information. Even small amounts of missing data can matter—especially if they are not random. So in most cases? **We prefer imputation over deletion.**

Mean Imputation — A Simple Example

Let's make this concrete. We have a dataset of workers:

WorkerID	Age	Salary
1	25	50000
2	NaN	54000
3	35	NaN
4	30	58000
5	24	45000

First, we compute the mean of each column (ignoring missing values):

- Mean $Age = (25 + 35 + 30 + 24)/4 = 28.5$
- Mean $Salary = (50000 + 54000 + 58000 + 45000)/4 = 51750$

Now we fill the missing values:

WorkerID	Age	Salary
1	25	50000
2	28.5	54000
3	35	51750
4	30	58000
5	24	45000

Simple. Clean. Done.

Median Imputation

Now let's try something more robust. Sort the values first.

- $Age \rightarrow 25, 30, 35 \rightarrow Median = 30$
- $Salary \rightarrow 45000, 50000, 54000, 58000 \rightarrow Median = 52000$

After imputation:

WorkerID	Age	Salary
1	25	50000
2	30	54000
3	35	52000
4	30	58000
5	24	45000

Notice how the median resists extreme values better than the mean.

Mode Imputation

Now consider frequency. The **mode** is the most common value.

Example (we expanded our dataset):

WorkerID	Age	Salary
1	25	50000
2	NaN	54000
3	35	NaN
4	30	58000
5	35	45000
6	20	50000
7	30	NaN
8	30	50000
9	35	NaN
10	25	58000

- Mode *Age* → 30 and 35
- Mode *Salary* → 50000

After filling:

WorkerID	Age	Salary
1	25	50000
2	30	54000
3	35	50000
4	30	58000
5	35	45000
6	20	50000
7	30	50000
8	30	50000
9	35	50000
10	25	58000

Useful for categorical data or repeated patterns.

The Important Truth (This is where most people get it wrong)

All these methods are **mechanical**. They follow rules. They apply formulas. But real data? Is not mechanical. It has context.

Let me show you what I mean. If someone's *Age* is missing, and their job title is: "**Senior Manager**". Do you really think their age is the dataset average? Probably not. And if *Salary* is missing? It likely depends on: *Age; Experience; Position; Industry*. Not just a global mean.

The Shift from "Filling Data" to "Understanding Data"

This is where beginners become practitioners. Instead of asking: "What number should I put here?" → You start asking: "What *should this value realistically be* given everything else?"

Maybe:

- Group by age range → compute average salary
- Use similar profiles → estimate missing values
- Use domain knowledge

Because in the end, **imputation is not about fixing data. It's about preserving reality.**

And that's the mindset you want to carry forward. Because as we move deeper into Machine Learning, you'll realize something: The model is only as smart as the data you prepare.

Imputation of Missing Values with Python

Let's move from theory to something real. Because understanding imputation is one thing. **Actually doing it in Python? That's where it becomes useful.**

Before we even write code, there is something I want you to keep in mind: Libraries like **Scikit-learn** *can* handle preprocessing. But just because it *can*, doesn't mean it *should*. Here's my recommendation:

| **Treat Data Preparation as a completely separate stage from Model Training.**

Even though it belongs to the Machine Learning pipeline, we don't mix them together blindly. Why? Because data cleaning is exploratory. It requires inspection and reasoning. It is not just a mechanical step. And for that? **Pandas is your best friend.**

Why Use Pandas for Imputation? Because Pandas is built for working with data. Not models. Not optimization. Not training. Just **data manipulation**.

It allows you to inspect missing values, compute statistics, apply transformations cleanly. And most importantly, you stay in control.

We use this simple dataset for demonstration:

dataset.csv

***(You can run the following codes in any Python environment to see the output.
Furthermore, you can remember these lines by heart)***

```
import pandas as pd

# Read the dataset
df = pd.read_csv('dataset.csv')

# Handle dataset
print("Dataset.csv")
print(df) # Print everything
print("\nDataset.csv (5 first lines)")
print(df.head()) # Print only the first 5 rows
print("\nDataset.csv (10 first lines)")
print(df.head(10)) # Print only the first 10 rows
```

```
# Get shape
row, column = df.shape
print("Number of rows:", row)
print("Number of columns:", column)
```

Number of rows: 19
Number of columns: 10

```
# Sum of rows and columns
print("Sum of columns:")
print(df.sum(axis=0)) # Or just df.sum()
print("\nSum of rows:")
print(df.sum(axis=1))
```

```
# Get the sum of all values
print("Sum of all values in the DataFrame:", df.sum().sum())
```

Sum of all values in the DataFrame: 3986.589

```
# Check if the data is NaN with df.isnull()
# df.isnull() returns a table (True if exists, False if not exists)
sum_of_null = df.isnull().sum().sum()
print("Total missing values:", sum_of_null)
```

Total missing values: 53

```
print("1) Fill in the missing values with mean.")
mean = df.mean() # or mean = df.sum() / df.count()
d_fillmean = round(df.fillna(mean), 3)
print(f"{d_fillmean}")
print("\n2) Fill in the missing values with median.")
median = df.median()
d_fillmedian = df.fillna(median)
print(f"{d_fillmedian}")
print("\n3) Fill in the missing values with mode.")
# In case there are multiple modes, iloc[0] selects the first mode found
mode = df.mode().iloc[0]
d_fillmode = df.fillna(mode)
print(f"{d_fillmode}")
```

```
# Drop all the rows with missing values
new_df = df.dropna()
print(new_df)
```

2. Training Data and Testing Data

Now you know how to handle missing values (encoding, and normalization will be in future sections), why don't we talk about Training and Testing Data?

This step is often overlooked because it feels automatic. Like something the machine learning library just *"handles for you"*. But do you even know why we need to split data in the first place? Let's slow down for a moment and think about what we're actually doing when we build a model.

When you give a model a dataset, you're not just storing information. You're asking it to learn patterns, relationships, and structure hidden inside that data. But here's the problem:

If the model sees everything, how do you know it actually learned anything? It's like giving someone the answer sheet before the exam. They might score perfectly—but that doesn't mean they understand the subject.

That's exactly why we split data into two parts:

- Training data → where the model learns
- Testing data → where we check if it actually learned something

Training Data — “The learning phase”

This is the portion of the dataset we *allow the model to study*. Typically around 70–80% of the data is used here. The model looks at this data and starts discovering patterns in features, relationships between inputs and outputs, and statistical regularities in the dataset

In other words, training data is where the model builds its internal “understanding”.

But there's an important catch: It can also memorize.

If we're not careful, the model may just store the training examples instead of learning general rules. That leads to overfitting—where the model performs extremely well on training data but fails on anything new.

Testing Data — “The reality check”

The remaining 20–30% of the dataset is held back completely. And “*held back*” here is strict. ***The model NEVER see the training data during training at all.*** This data is used only after training is finished, to answer one simple question:

Can this model handle new, unseen data?

We feed the testing data into the model, get predictions, and compare them with the real answers. This gives us a fair measure of performance.

A common metric is accuracy:

$$\text{Accuracy} = (\text{Correct Predictions} / \text{Total Predictions}) \times 100$$

But the key idea is not the formula—it's the fairness of the evaluation.

Why this split is necessary

Without splitting, you lose the ability to judge generalization. If a model sees all data during training, it can “memorize” answers. Performance looks perfect, but real-world performance collapses

This is why training without testing is meaningless in practice. We don't care how well a model remembers—we care how well it *understands unseen situations*.

Imagine building a house price prediction model:

- Training data = houses the model studies to understand pricing patterns
- Testing data = completely new houses used to check if those patterns actually work

If the model only memorized the training houses, it would fail the moment it sees a new one.

But if it learned relationships like: bigger houses tend to cost more; location heavily affects price; number of rooms has partial influence. Then it will perform well on unseen houses too.

That's the real goal. Training data teaches the model. Testing data exposes whether the model actually learned, or just memorized.

And that simple separation is what turns machine learning from "data fitting" into real prediction systems. Because in the end, we don't care how well a model performs on what it has already seen. We care how it behaves in the unknown.

Training Data and Testing Data with Python

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Read the dataset
df = pd.read_csv('dataset.csv')
# Split the data into 80% for training and 20% for testing
df_train, df_test = train_test_split(df, test_size=0.2, random_state=42)
# Return the shape of the training and testing dataset
print(f"Training Shape: {df_train.shape}")
print(f"Testing Shape: {df_test.shape}")
```

Training Shape: (15, 10)

Testing Shape: (4, 10)

3. Validation Data — "The Mock Exam for the Model"

Now here's where things get a little more interesting. So far, we've split our data into two roles: training data for learning, and testing data for final judgment. But in practice, we often need something in between. That's where **validation data** comes in.

Think of it like this: If training data is your study material, and testing data is the final exam, then validation data is your **mock exam**. It's not used to teach the model. And it's

not used to judge it at the very end either. Instead, it quietly sits in the middle of the process—helping us make decisions while the model is still learning.

During training, we constantly check the model's performance on validation data. Not to make it memorize anything, but to guide it. For example:

- Is the model learning properly, or starting to overfit?
- Should we stop training early?
- Which settings (hyperparameters) actually work better?

So validation data becomes a kind of feedback loop. It tells us: ***"You're improving... or you're overfitting... or you're not learning enough yet."***

And here's the key difference:

- **Training data** → teaches the model
- **Validation data** → helps tune the model
- **Testing data** → gives the final, unbiased verdict

Even though validation data is not used to directly train the model, it still influences how the model evolves. It's like a coach watching practice sessions and giving feedback, without playing the game itself.

That's why in many real-world systems, we split data into three parts:

- **Training set ($\approx 60\text{--}70\%$)** → learning patterns
- **Validation set ($\approx 10\text{--}20\%$)** → tuning and decisions
- **Testing set ($\approx 10\text{--}20\%$)** → final evaluation

If validation performance is very different from testing performance, that's a red flag. It usually means the model was too "optimized" for the mock exam and didn't truly generalize.

In short, validation data is where we *shape* the model, before we finally *judge* it.

Validation Data with Python

```

import pandas as pd
from sklearn.model_selection import train_test_split

# Read the dataset
df = pd.read_csv('dataset.csv')
# Split the data into 80% for training and 20% for validation/testing
df_train, df_val_test = train_test_split(df, test_size=0.2,
random_state=42)
# From the validation/testing data, split 10% to validation data,
and 10% for testing
df_val, df_test = train_test_split(df_val_test, test_size=0.5, random_state=42)
# Return the shape of the training and testing dataset
print(f"Training Shape: {df_train.shape}")
print(f"Validation Shape: {df_val.shape}")
print(f"Testing Shape: {df_test.shape}")

```

Training Shape: (15, 10)

Validation Shape: (2, 10)

Testing Shape: (2, 10)

Summary

So let's connect everything together. At the core of this section, we learned that building a machine learning model is not just about feeding data and getting results. It's about **how we manage data during learning**.

We start by handling missing values. Missing values can prevent a model from training consistently. Missing values can be imputed using mean, median, mode, or completely removed from the data. Or you can train a machine learning model with known samples to predict the missing ones.

We then split data into different roles:

- **Training data** lets the model learn patterns from examples.
- **Validation data** acts as a checkpoint during training, helping us adjust and improve the model.
- **Testing data** is kept completely untouched until the very end, where it measures real-world performance.

This separation is important because it protects us from a very common problem:

Overfitting. A model that performs perfectly on training data might just be memorizing instead of understanding. And validation/testing help us catch that early.

So the real idea is not just “train and test”, but to carefully structure the learning process so that the model doesn’t just remember, but actually generalizes.

In simple terms: Training teaches the model, validation guides it, and testing proves whether it truly learned.

And that’s the foundation of building models that don’t just work in theory—but actually survive in the real world.

References

<https://www.geeksforgeeks.org/data-analysis/what-is-data-preparation/>

https://www.tutorialspoint.com/machine_learning/machine_learning_preparing_data.htm

<https://developers.google.com/machine-learning/clustering/prepare-data>

Next Section:

 [2. Data Normalization Methods](#)